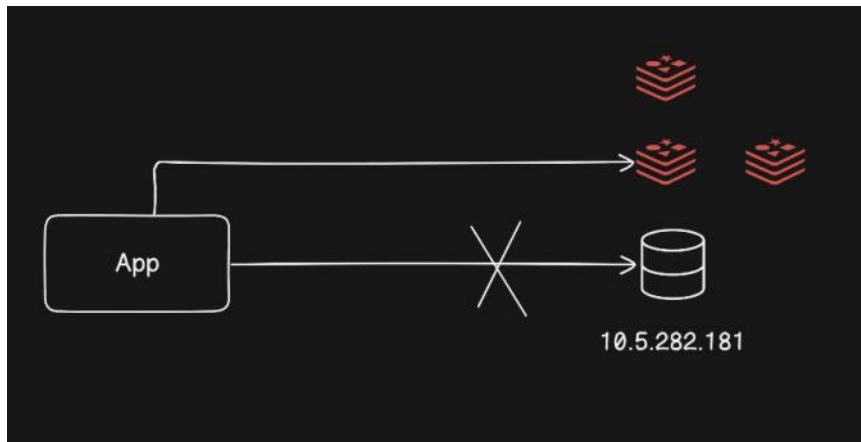


Exception Handling

04 July 2026 09:09

In real application problem will happen.
User may enter wrong input
File may not exist
Database may be down



Exception in thread "main" java.lang.ArithmeticException: / by zero at Test.main(Test.java:9)

The program stopped at a/b
The next line did not execute
This runtime problem is called exception

What is Exception ?

An exception is an unwanted or unexpected event that happens during the program execution and disturbs the normal flow of the program.

Exception = runtime problem in the program

Error : usually represents a serious problem that app code normally cannot handle
exception represents a problem that app code can handle.

Note:

As a backend developer, we mainly handle **exceptions**, not errors.

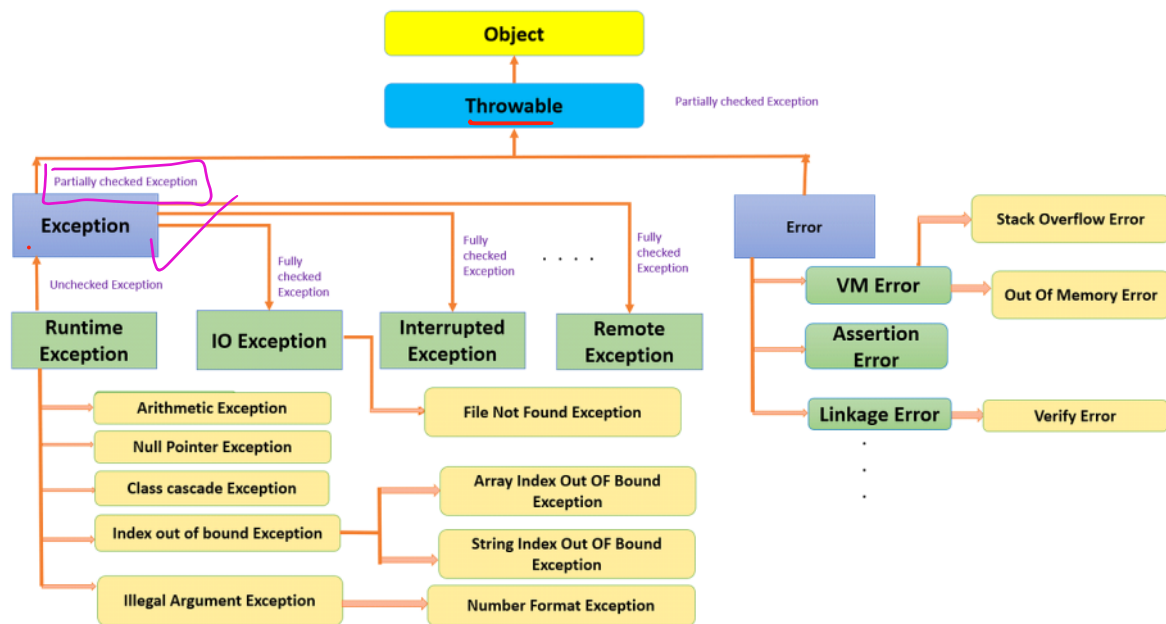


Fig. Exception Hierarchy in Java ~ by Deepthi Swain

Type of exception

1. Checked Exception

Checked exceptions are those that the compiler force you to handle explicitly.
Eg: IOException, SQLException

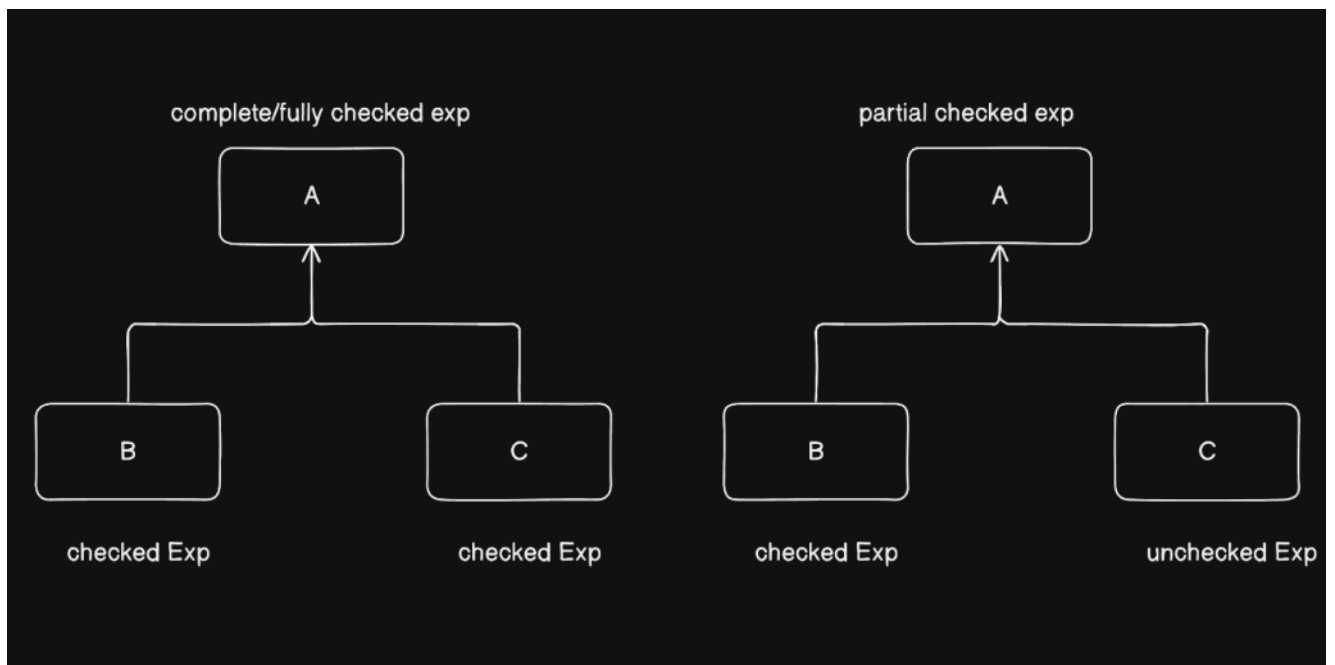
Handling checked exception is done by using try-catch block

2. Unchecked Exception

In Unchecked Exception compiler will not force you to handle explicitly but thrown at runtime.

Eg: NullPointerException, ArrayIndexOutOfBoundsException

Note: RuntimeException and it's all child classes , Error and it's child classes are unchecked and all remaining are considered as checked exception

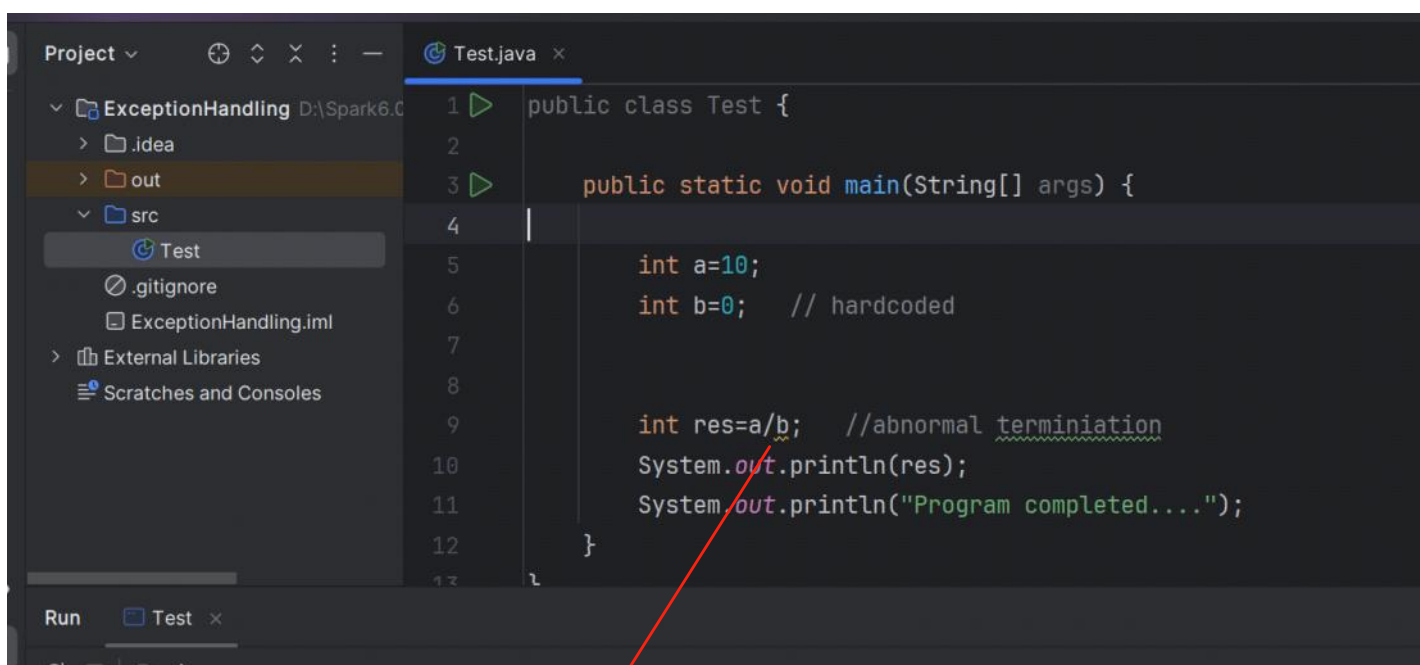


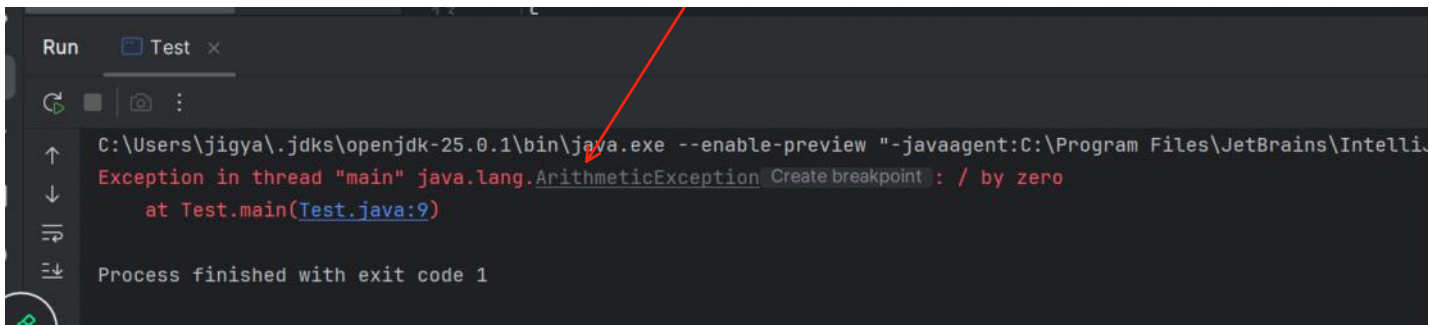
Compile Time Error Vs Runtime Exception

Compile time error: this happens before program runs.

```
1 public class Test {  
2  
3     public static void main(String[] args) {  
4  
5         int age="twenty";  
6     }  
7 }  
8
```

Runtime Exception:





```
Run Test x
C:\Users\jigya\.jdk\openjdk-25.0.1\bin\java.exe --enable-preview "-javaagent:C:\Program Files\JetBrains\IntelliJ...
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Test.main(Test.java:9)
Process finished with exit code 1
```

Why do we need Exception handling ?

Without exception handling:

1. Program stop suddenly
2. Users sees technical error
3. Remaining code does not execute
4. Application may give bad experience

With exception handling

1. Program can show meaningful message
2. Remaining code can continue
3. Developer can log the real program

Real-world examples:

Suppose enter wrong password

Application should not crash, it should show: **Invalid username or password**

Suppose Database is down.

Application should not expose full java error to user.

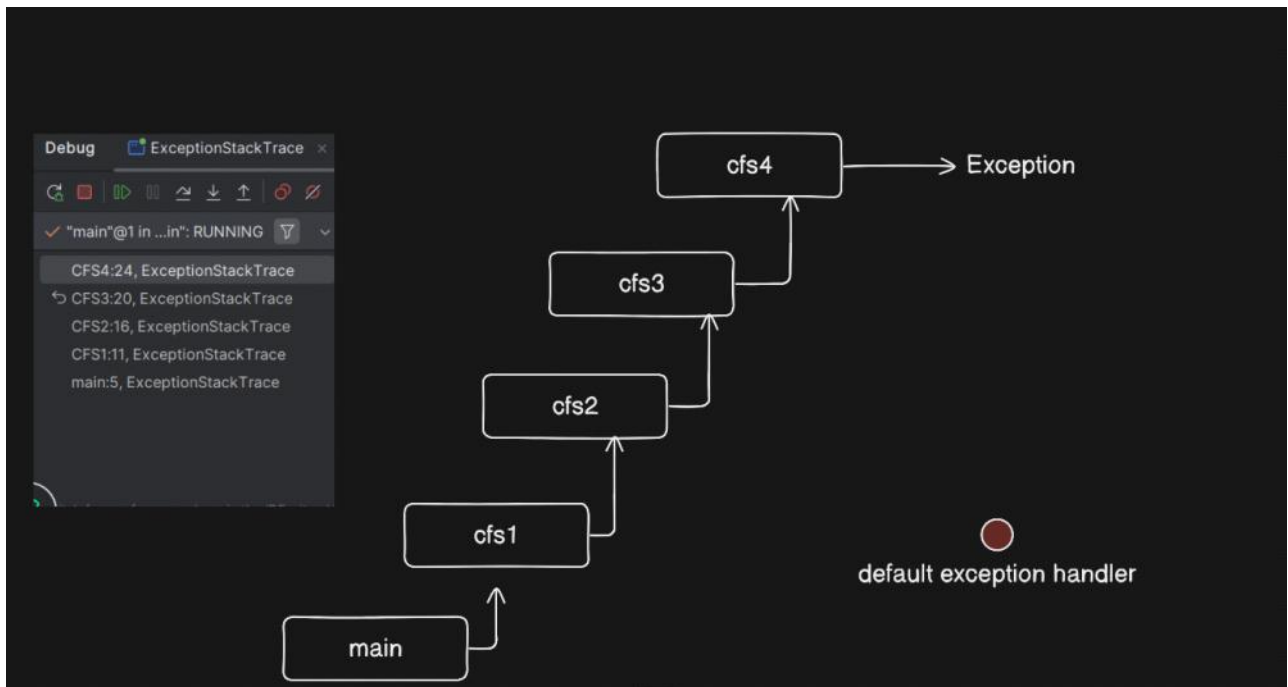
It should return a proper response like: **"Service temporarily unavailable"**

Exception handling keywords

Java mainly gives these keywords:

1. Try-Catch
2. Finally
3. Throw
4. Throws

Keyword	Description
Try	Write Risky code
Catch	Handle exception
Finally	Always execute cleanup code
Throw	Manually create/raise exception
Throws	Delicate responsibility



Various method to print information

1. `printStackTrace():`

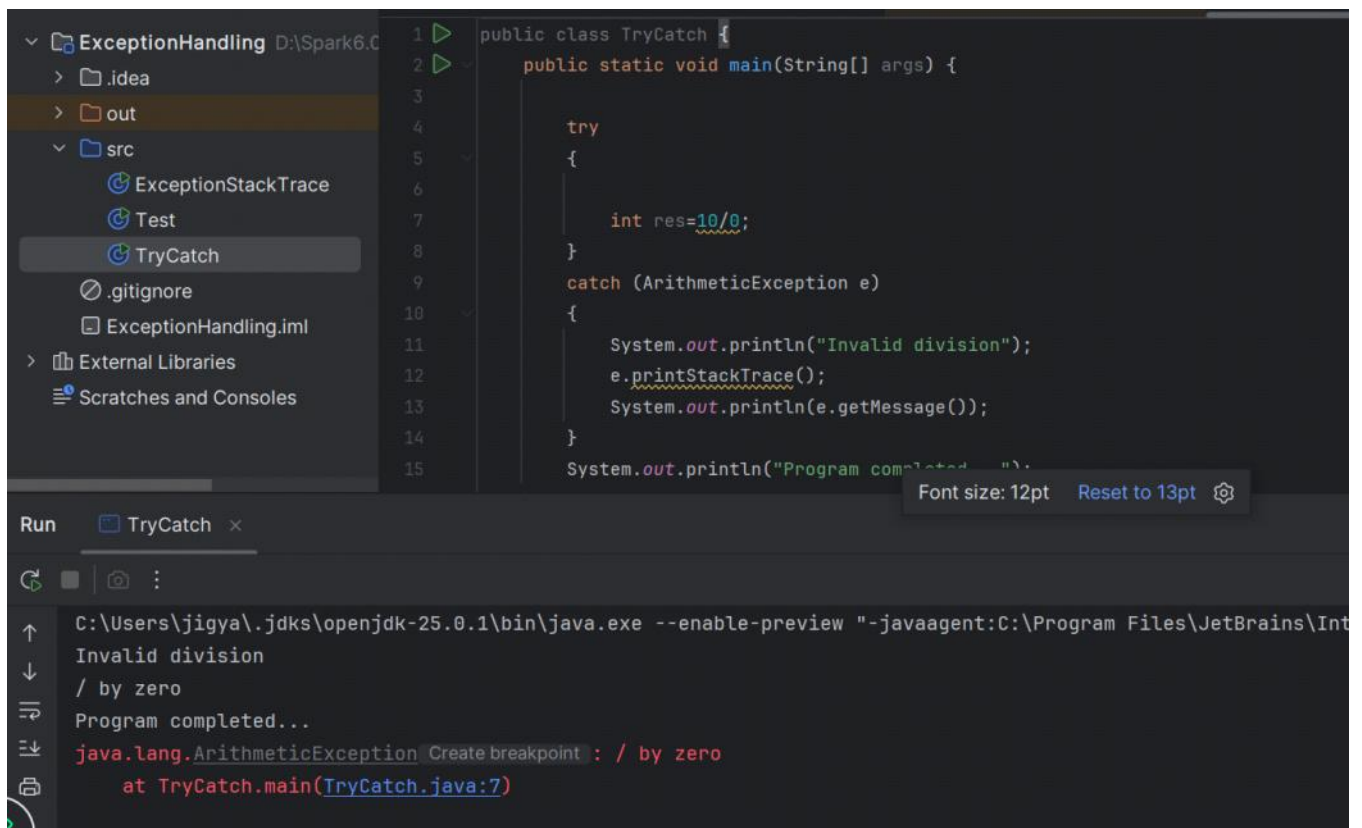
This method print exception information in following format. Name of the exception:
description of exception stack trace

2. `toString()`

This method print exception information in the following format. Name of the exception:
Description of exception

3. `getMessage()`

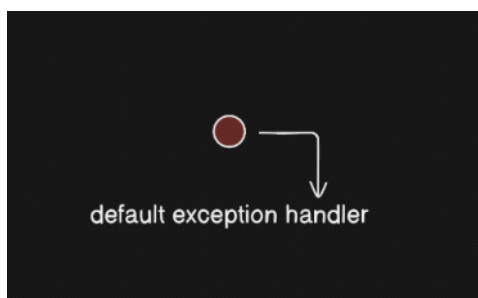
This method return only description of the exception



Flow of try-catch

Finally

Finally block contains code that should execute whether exception occurs or not.



throw:

Throw is used to manually throw an exception

Custom exception

V.V.V.V.V.V.V.IMP : how to create custom exception